

Size Change Termination Condition for Injective Relations

Stefano Berardi

April, 2025

1 Introduction

1.1 Introduction to Size Change Termination

Size-change termination (SCT), initially proposed by Lee et al. [LJBA01], is a technique for proving the termination of recursive functions. SCT leverages Fermat's principle of infinite descent: to prove a proposition P , assume its negation and derive a contradiction by constructing an infinite descending chain within a well-ordered set. In the context of recursive functions, if the function's variables are interpreted within a well-ordered domain, non-termination can be assumed, and a contradiction derived by demonstrating an impossible infinite descending sequence of variable values. These sequences are formed by analyzing the relationships between variables across recursive calls. Each relationship is abstracted as a *size-change graph*, which represents variable transformations during program execution. To illustrate this concept, consider the following example of the Ackermann function, taken from [Lee09]:

```
a(m,n) = if m=0 then n+1 else
         if n=0 then 1a(m-1,1) else
         2a(m-1, 3a(m,n-1))
```

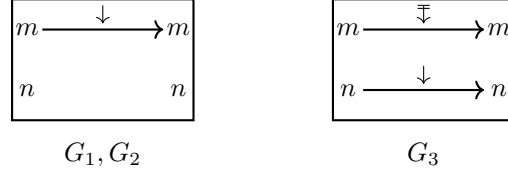


Figure 1: Definition of the Ackermann function a and size change graphs G_1 , G_2 and G_3

For each recursive call, labeled with 1,2 and 3, we can respectively define the size change graphs G_1 , G_2 and G_3 , where the edges $x \xrightarrow{\gamma} y$ represent the relationship between the values of the variables initially passed to the function, and the values passed in each recursive call. When $\gamma = \downarrow$, we call it a *progressing* edge and it specifies that the value of x is strictly less than the value of y . When $\gamma = \Downarrow$, we call it a *stationary* edge, and it specifies that the value of x is less than or equal to the value of y .

Using these graphs we can then analyze the function's execution: assume a hypothetical infinite execution from an initial call of the program a . This will induce an infinite sequence of recursive calls $c = c_1 c_2 c_3 \dots$ where every $c_i \in \{1, 2, 3\}$, which also induces an infinite sequence of size change graphs $G_{c_1} G_{c_2} G_{c_3} \dots$. It is then possible to compose these graphs, to form all the infinite paths using the relations between each one of the variables, named as a *mutipath*.

For example, suppose we get an infinite execution $(13)^\omega$, i.e. the recursive call 1 and 3 are used infinitely one after the other. This induces the sequence of graphs $(G_1 G_3)^\omega$, seen as a multipath in Figure 2. We can then identify the infinite path $m \xrightarrow{\downarrow} m \xrightarrow{\Downarrow} m \xrightarrow{\downarrow} m \dots$ over the variable m , which progresses infinitely often and implies that this infinite execution is impossible, since the domain of the variable m is $\mathbb{N}$. Given this intuition, we can generally define the SCT property programs as follows:

Definition 1.1. (Size Change Termination) A program satisfies the SCT if and only if for any possible infinite sequence of recursive calls $c = c_1 c_2 c_3 \dots$, the induced composition of infinite size change graphs $G_{c_1} G_{c_2} G_{c_3} \dots$ has an infinite path that infinitely progresses.

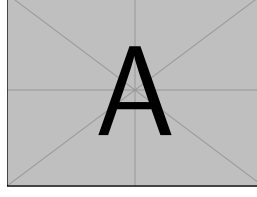


Figure 2: Multipath for $(13)^\omega$

1.2 Introduction to Global Trace Condition

The *Global trace condition* (GTC) is used to verify the validity of *circular pre-proofs* in *circular proof systems*. In this case, we are considering the general notion of the global trace condition, which is independent of the proof system, as presented in Brotherston's [Bro06], or Afshari and Wehr's [AW24] work (in the case of latter, we consider the $\mathbb{B}$ -activated trace category $\mathcal{T}_{\mathbb{B}}$, where $\mathbb{B} = \{\top, \perp\}$ is the binary boolean algebra). For the sake of simplicity, we will rely on Brotherston's work to introduce this concept.

Let CS^ω be a proof system, with Seqs the set of well-formed sequents in CS^ω , and Rules be the set of logical rules in CS^ω . A *circular pre-proof* is represented by a tuple $\mathcal{P} = (\mathcal{D}, \mathcal{R})$. The *derivation graph* $\mathcal{D}$, is a tuple $\mathcal{D} = (V, s, r, p)$, where V is a set of vertices, $s : V \rightarrow \text{Seqs}$ is total function that labels vertices with sequents, $r : V \rightarrow \text{Rules}$ is partial function that labels vertices with inference rules in CS^ω , and $p : \mathbb{N} \times V \rightarrow V$ is a partial function that maps a number n and a vertex v , with a vertex v' , such that v' is labeled by the n th premise rule $r(v)$ over the sequent $s(v)$. The idea then is that for every $v \in V$, the following inference is an instance of the rule $r(v)$ in CS^ω , with m premises:

$$\frac{\{\vdash s(p(i, v))\}_{1 \leq i \leq m}}{\vdash s(v)} (r(v))$$

Moreover, a *bud* node of $\mathcal{D}$ is a vertex $B \in V$ such that $r(B)$ is undefined. The partial function $\mathcal{R} : V \rightarrow V$, maps each one of the bud nodes to an internal node in the derivation graph, called a *companion*. To understand the intuition behind these definitions, we will use an example of a circular pre-proof in the CLKID^ω proof system, taken from [Bro06]. In this system, we allow the definition of inductive predicates, and we will assume that the variables are interpreted in $\mathbb{N}$. We represent inductively each natural number, as either 0, or using the successor function s . We define the rules for the inductive predicates N, P and Q in Figure 3. P and Q are mutually dependent inductive predicates, while N represents the usual definition of the natural numbers.

$$\begin{array}{c} \frac{}{\Gamma \vdash P0, \Delta} (PR_1) \qquad \frac{\Gamma \vdash Px, \Delta \quad \Gamma \vdash Q(x, sx), \Delta}{\Gamma \vdash Psx, \Delta} (PR_2) \\[10pt] \frac{}{\Gamma \vdash Q(x, 0), \Delta} (QR_1) \qquad \frac{\Gamma \vdash Q(x, y), \Delta \quad \Gamma \vdash Px, \Delta}{\Gamma \vdash Q(x, sy), \Delta} (QR_2) \\[10pt] \frac{\Gamma, x = 0 \vdash \Delta \quad \Gamma, x = sz, Nz \vdash \Delta}{\Gamma, Nx \vdash \Delta} (\text{Case } N) \end{array}$$

Figure 3: Rules in CLKID^ω , for the unary predicates N, P and the binary predicate Q

Figure 4, presents an example of a circular pre-proof in the CLKID^ω proof system, that uses the rules defined for the inductive predicates P, Q and N , and other well known structural rules like contraction or weakening, and rules associate to first order proof systems like substitution and variable renaming. The derivation graph is given by the tree structure observed in Figure 4 and the bud nodes correspond to the leaves in the tree that have no rule associated, e.g. the leaves labeled with the sequent $Nx, Ny \vdash Q(x, y)$. The function $\mathcal{R}$, is represented by labeling each bud and companion node with the same symbol, e.g. the

buds labeled with $(\dagger_1)$ and $(\dagger_2)$ map to the companion labeled with $(\dagger)$, and similarly for the vertices labeled with $(*)$.

$$\begin{array}{c}
\frac{Nx \vdash Px (*) \text{ (Subst)} \quad \frac{Ny, Nx \vdash Q(x, y) (\dagger_2) \text{ (Subst)}}{Nsz, Nz \vdash Q(z, sz)} \text{ (PR}_2\text{)}}{Nz \vdash Pz} \text{ (Subst)} \\
\frac{\frac{Nx \vdash Px (*) \text{ (Subst)}}{Nz \vdash Pz} \text{ (Subst)} \quad \frac{Ny, Nx \vdash Q(x, y) (\dagger_2) \text{ (Subst)}}{Nsz, Nz \vdash Q(z, sz)} \text{ (PR}_2\text{)}}{Nsz, Nz \vdash Psz} \text{ (=L)} \\
\frac{\frac{Nx \vdash Px (*) \text{ (Subst)}}{Nz \vdash Pz} \text{ (Subst)} \quad \frac{Ny, Nx \vdash Q(x, y) (\dagger_2) \text{ (Subst)}}{Nsz, Nz \vdash Q(z, sz)} \text{ (PR}_2\text{)}}{Nx, x = sz, Nz \vdash Px} \text{ (Case N)} \\
\frac{\frac{Nx, Ny \vdash Q(x, y) (\dagger_1) \text{ (Subst)}}{Nx, Nz \vdash Q(x, z)} \text{ (Subst)} \quad \frac{Nx, Nz \vdash Px}{Nx \vdash Px (*)} \text{ (Contr L)}}{Nx, Nz \vdash Q(x, sz)} \text{ (=L)} \\
\frac{Nx \vdash Q(x, 0) \text{ (QR}_1\text{)}}{Nx, y = 0 \vdash Q(x, y)} \text{ (=L)} \quad \frac{Nx, Nz \vdash Q(x, sz)}{Nx, y = sz, Nz \vdash Q(x, y)} \text{ (=L)} \\
\frac{Nx, y = 0 \vdash Q(x, y) \quad Nx, y = sz, Nz \vdash Q(x, y)}{Nx, Ny \vdash Q(x, y) (\dagger)} \text{ (Case N)}
\end{array}$$

Figure 4: Example of a circular proof in CLKID^ω of the sequent $Nx, Ny \vdash Q(x, y)$. The double horizontal lines in some of the rules represent a hidden weakening.

It is then possible to extract the graph structure of $\mathcal{P}$, written as $\mathcal{G}_{\mathcal{P}}$, by adding the edges induced by $\mathcal{R}$ and deleting the bud vertices. Formally, $\mathcal{G}_{\mathcal{P}} = (V', s, r, p')$ where $V' = V \setminus \text{Bud}(\mathcal{D})$ and p' is defined for some $j \in \mathbb{N}$, and $v \in V$ as $p'_j(v) = \mathcal{R}(p_j(v))$ if $p_j(v) \in \text{Bud}(\mathcal{D})$, and $p'_j(v) = p_j(v)$ otherwise. A *path* in $\mathcal{G}_{\mathcal{P}}$, is then a finite or infinite sequence of vertices over $\mathcal{G}_{\mathcal{P}}$. Moreover, given a (possibly infinite) path π , over a graph $\mathcal{G}_{\mathcal{P}}$, it is possible to form (possibly infinite) sequences of logical formulas called *traces*.

A trace $\tau = F_1 F_2 F_3 \dots$ over a path $\pi = v_1 v_2 v_3 \dots$ must satisfy that for all i , F_i belongs to the sequent $s(v_i)$, and F_{i+1} must be a direct ancestor of F_i , after applying the rule $r(v_i)$. For example, let v be the vertex labeled with $(\dagger)$ and v' be the vertex labeled with the sequent $Nx, Nz \vdash Q(x, z)$. We can then form the infinite path $\pi = v \dots v' v \dots v' v \dots$ over $\mathcal{G}_{\mathcal{P}}$ which starts at v and infinitely travels through the cycle C_1 , as seen in Figure 5. We can then form the infinite trace $\tau = Ny, Nz, Nz, Nz, Ny \dots$, to cover the path π .

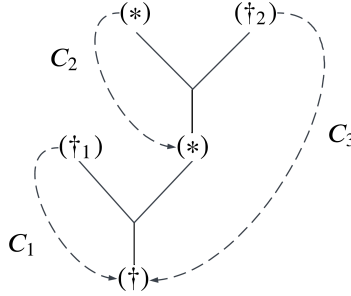


Figure 5: Abstraction of the graph structure of the Figure 4. Each basic cycle C_i with $i \in \{1, 2, 3\}$, corresponds to the unique path from each companion to itself, using the induced edge by its corresponding bud node.

Moreover, we define a progress point (F_i, F_{i+1}) of a trace τ , as a point where the values of the variables in formula F_i are strictly less than the value of the variables in formula F_{i+1} . For example, in the case of the previously defined trace $\tau = Ny, Nz, Nz, Nz, Ny \dots$, (Ny, Nz) is a progress point, since $y > z$. In fact, the trace τ contains infinitely many progress points (Nz, Ny) . Note that for the sake of simplicity, the definition of trace and progress point are oversimplifications of the formal notion of *generalized trace and progress point* introduced by Brotherston in [Bro06], section 4.2.1. The curious reader is encouraged to review this definition, but for the moment we would not dive into the technicalities, since our main idea is only to provide a general intuition.

Using all of the previous definitions, it is now possible to define the GTC as follows:

Definition 1.2. (Global Trace Condition) Let $\mathcal{P}$ be a CS^ω circular pre-proof. We say that $\mathcal{P}$ satisfies the GTC or $\mathcal{P}$ is a CS^ω circular proof, if and only if every infinite path π in $\mathcal{G}_{\mathcal{P}}$ has a trace τ with infinitely many progress points.

In the case of the circular pre-proof in Figure 4, it is easy to see that for any infinite path we can find an infinitely progressing trace. Furthermore, the GTC also uses Fermat’s principle of infinite descent, in the sense that every infinite path in the pre-proof’s graph has an infinitely descending chain of natural numbers, contained in the path’s trace, and thus, these paths can be seen as logical reasonings that can be disregarded, since they lead to a contradiction.

1.3 Generalized SCT and GTC

1.4 MISC

In this case, we are considering the general notion of the global trace condition, which is independent of the proof system, as presented in Brotherston’s [Bro06], or Afshari and Wehr’s [AW24] work (in the case of latter, we consider the $\mathbb{B}$ -activated trace category $\mathcal{T}_{\mathbb{B}}$, where $\mathbb{B} = \{\top, \perp\}$ is the binary boolean algebra). For the sake of simplicity, we will rely on Brotherston’s work to introduce this concept.

We are going to change the narrative:

1. We define and explain the generality of the size change termination condition (def + example).
2. We define and explain the generality of the global trace condition (def + example)
3. We remark that both of them are equivalent and can be seen as one definition given an abstract framework using only relations and the underlying structure of the program/proof tree
4. Then we introduce PDC with injectivity
5. We argue why circular proofs are in general injective
6. We explain what are the advantages of using PDC + Injectivity over the abstract version of STC/GTC

References

- [AW24] Bahareh Afshari and Dominik Wehr. Abstract cyclic proofs. *Mathematical Structures in Computer Science*, 34(7):552–577, 2024.
- [Bro06] James Brotherston. Sequent calculus proof systems for inductive definitions. 2006.
- [Lee09] Chin Soon Lee. Ranking functions for size-change termination. *ACM Trans. Program. Lang. Syst.*, 31(3), April 2009.
- [LJBA01] Chin Soon Lee, Neil D. Jones, and Amir M. Ben-Amram. The size-change principle for program termination. In *Proceedings of the 28th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL ’01, page 81–92, New York, NY, USA, 2001. Association for Computing Machinery.